

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

PHASE 2 — FINAL SESSION



Session 16

Kubernetes Advanced

HPA · Resource Limits · Helm Charts · RBAC · Phase 2 Retrospective



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY

Monday

TIME

3:00–5:30 PM

DURATION

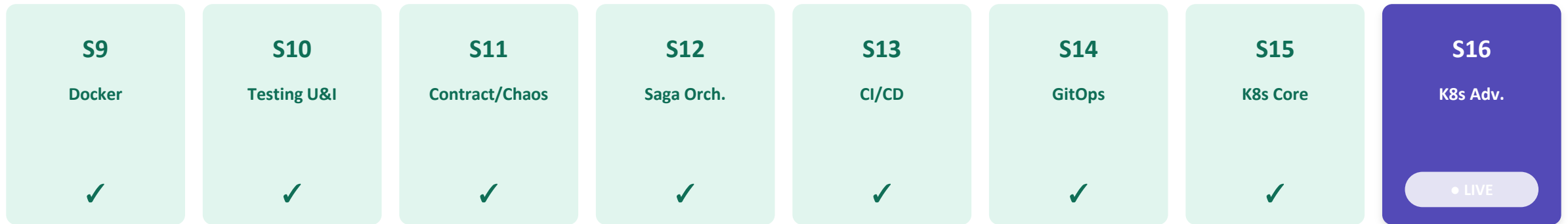
2.5 Hours

PHASE

Phase 2 Final · S16/29

Where We Are — Phase 2 Final Session

This is it — the last session of Phase 2. After today, the delivery pipeline is complete end-to-end.



Recap: S15 gave product-service health signals (probes) and a Secret for the DB password.

Today: make the platform scale itself under load, and package deployments so they're reusable across environments — with least-privilege access baked in.

Traffic Spike → OOMKill → Cascading Failure

- 1 Product launch day. Traffic to product-service increases 10x in 5 minutes.
- 2 2 pods. Each requests 256Mi memory. Each has a limit of 256Mi.
- 3 One pod tries to load a large product catalog into cache: 350Mi. LIMIT EXCEEDED.
- 4 K8s OOMKills the pod. 50% of traffic suddenly goes to 1 pod instead of 2.
- 5 The remaining pod hits 100% CPU. Requests timeout. P99 latency: 8 seconds.
- 6 **No HPA configured. Nobody adds pods. The incident lasts 22 minutes.**

Three compounding failures: (1) limits too tight → OOMKill, (2) no HPA → can't scale out, (3) requests not set → K8s couldn't schedule correctly.

Resource Requests vs Limits

	Resource Requests	Resource Limits
Purpose	Scheduling guarantee — node has this much available	Hard cap — CPU throttled, Memory OOMKilled
If not set	Pod scheduled anywhere (uncontrolled)	Pod has unlimited access (noisy neighbor)
CPU unit	millicores: 100m = 0.1 CPU	millicores: 500m = 0.5 CPU
HPA interaction	HPA calculates: actual CPU / requested CPU	HPA ignores limits entirely

Resource Block for a Spring Boot Microservice

```
resources:
  requests:
    cpu: 200m          # 0.2 CPU – scheduler hint
    memory: 512Mi      # 512 MiB – Spring Boot + JVM baseline
  limits:
    cpu: 1000m         # 1 CPU max – prevent CPU hogging
    memory: 768Mi      # allow JVM heap to grow

# WARNING: memory limit must be > JVM heap size
# Set -Xmx to 70% of memory limit to leave headroom for OS/JVM overhead
# Example: limit 768Mi → -Xmx538m

# DEV ONLY: relaxed limits below for local cluster (minikube 2-node):
# requests: {cpu: 100m, memory: 256Mi}
# limits: {cpu: 500m, memory: 512Mi}
```

HPA — The CPU Feedback Loop



SCALE OUT: CPU utilization $\geq 70\%$ for 30s → add pods

SCALE IN: CPU utilization $< 50\%$ for 300s (5 min cooldown) → remove pods — prevents thrashing

HPA — The CPU Feedback Loop

```
Metrics Server (reads actual CPU) → HPA Controller
|
| calculates: avg CPU utilization across pods
| if utilization > targetCPUUtilizationPercentage:
|   desired replicas = ceil(currentReplicas × utilization/target)
|
▼
Deployment
(update replica count)
|
▼
New Pods start
(readiness probe gates traffic → S15)
```

SCALE OUT: CPU utilization $\geq 70\%$ for 30s $\rightarrow$ add pods

SCALE IN: CPU utilization $< 50\%$ for 300s (5 min cooldown) $\rightarrow$ remove pods

The cooldown prevents thrashing: don't remove pods immediately after a spike subsides.

HorizontalPodAutoscaler for product-service

```
# k8s/product-service/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: product-service-hpa
  namespace: ecommerce
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: product-service
  minReplicas: 2          # always run minimum 2 for availability
  maxReplicas: 10         # hard cap – prevent runaway scaling costs
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: {type: Utilization, averageUtilization: 70} # scale out when avg CPU ≥ 70% of requested
# DEV NOTE: HPA requires Metrics Server running in the cluster.
# minikube: minikube addons enable metrics-server
# kind: kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/...
```

Watching HPA React to Load

```
$ kubectl get hpa -n ecommerce
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
product-service-hpa	Deployment/product...	35%/70%	2	10	2

```
# ... after a load test ...
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
product-service-hpa	Deployment/product...	140%/70%	2	10	4

HPA requires Metrics Server running in the cluster. minikube: minikube addons enable metrics-server

Raw K8s Manifests vs Helm Charts

1-2 services, team owns all manifests?

→ Raw manifests (simpler)

5+ services with shared config values?

→ Helm (parametrize once, reuse everywhere)

Need deployment version rollback?

→ Helm (helm rollback tracks history)

Team new to K8s?

→ Raw manifests first — learn primitives

Multiple environments (dev/staging/prod)?

→ Helm (different values.yaml per env)

ArgoCD manages deployments?

→ Either works — Helm is a natural fit

One-off debug deployment?

→ Raw manifests (kubectl apply)

Helm Chart Structure

Our course: raw manifests in S14-S15 (learn the primitives). Helm in S16 (package them). Both coexist in the repo.

```
# Create the Helm chart for product-service:
helm create product-service-chart

# Generated structure (we will customize it):
product-service-chart/
├── Chart.yaml           # chart metadata (name, version)
├── values.yaml          # default configurable values
└── templates/
    ├── deployment.yaml  # uses {{ .Values.* }} for parametrization
    ├── service.yaml
    ├── configmap.yaml
    ├── secret.yaml
    ├── hpa.yaml         # from earlier in this session
    ├── serviceaccount.yaml # RBAC foundation
    └── role.yaml         # RBAC permissions
```

Chart.yaml

```
# product-service-chart/Chart.yaml
apiVersion: v2
name: product-service
description: Product Service Helm Chart — E-Commerce Platform
type: application
version: 0.1.0      # chart version (increment when chart structure changes)
appVersion: "1.0.0" # application version (informational — actual version in values.yaml)
```

values.yaml — The Parametrization Contract

```
# product-service-chart/values.yaml
image:
  repository: ghcr.io/org/product-service
  tag: latest      # CI overrides: helm upgrade --set image.tag=sha-abc1234
replicaCount: 2    # initial replicas (HPA will override in practice)
service: {type: ClusterIP, port: 8081}
hpa:
  enabled: true
  minReplicas: 2
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70

resources:
  requests: {cpu: 200m, memory: 512Mi}
  limits: {cpu: 1000m, memory: 768Mi}

serviceAccount: {create: true, name: product-service-sa}

namespace: ecommerce
```

templates/deployment.yaml — Go Template Expressions

```
# product-service-chart/templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
  namespace: {{ .Values.namespace }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: product-service
  template:
    metadata:
      labels:
        app: product-service
```

templates/deployment.yaml — Go Template Expressions - Continue

```
spec:
  serviceAccountName: {{ .Values.serviceAccount.name }}    # ← RBAC link
  containers:
    - name: product-service
      image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
      imagePullPolicy: {{ .Values.image.pullPolicy }}
      ports:
        - containerPort: {{ .Values.service.port }}
      resources:
        requests:
          cpu: {{ .Values.resources.requests.cpu }}
          memory: {{ .Values.resources.requests.memory }}
        limits:
          cpu: {{ .Values.resources.limits.cpu }}
          memory: {{ .Values.resources.limits.memory }}
```

templates/deployment.yaml — Go Template Expressions - Continue

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: {{ .Values.service.port }}
  initialDelaySeconds: 60
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: {{ .Values.service.port }}
  initialDelaySeconds: 30
  periodSeconds: 5
```

The Chart Needs a ServiceAccount

“The chart deploys product-service as a pod. That pod may need to call the K8s API to read its own ConfigMap at runtime. We need a ServiceAccount and permissions — watch how the chart includes RBAC automatically.”



Without a dedicated SA

Pod runs as 'default' ServiceAccount — may have broad cluster permissions (dangerous if compromised)



With product-service-sa

Pod can ONLY read ConfigMaps/Secrets in its namespace. Compromise ≠ cluster write access.

ServiceAccount + Role — Read-Only, Least Privilege - ServiceAccount

```
# product-service-chart/templates/serviceaccount.yaml
{{- if .Values.serviceAccount.create }}
apiVersion: v1
kind: ServiceAccount
metadata:
  name: {{ .Values.serviceAccount.name }}
  namespace: {{ .Values.namespace }}
  labels:
    app: product-service
{{- end }}
```

ServiceAccount + Role — Read-Only, Least Privilege - Role

```
# product-service-chart/templates/role.yaml
{{- if .Values.serviceAccount.create }}
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: product-service-role
  namespace: {{ .Values.namespace }}
rules:
- apiGroups: [""]
  resources: ["configmaps", "secrets"]
  verbs: ["get", "list"]           # read-only - least privilege
  # NOT: create, update, delete, patch - product-service only READS config
```

ServiceAccount + Role — Read-Only, Least Privilege - RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: product-service-rolebinding
  namespace: {{ .Values.namespace }}
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: product-service-role
subjects:
- kind: ServiceAccount
  name: {{ .Values.serviceAccount.name }}
  namespace: {{ .Values.namespace }}
{{- end }}
```

Install → Upgrade → Rollback

```
# First install
$ helm install product-service ./product-service-chart -n ecommerce

# Upgrade with new image tag (CI/CD does this after push):
$ helm upgrade product-service ./product-service-chart \
  --set image.tag=sha-abc1234 -n ecommerce

# List releases:
$ helm list -n ecommerce
```

NAME	NAMESPACE	REVISION	STATUS	CHART
product-service	ecommerce	2	deployed	product-service-0.1.0

```
# Rollback to previous release:
$ helm rollback product-service 1 -n ecommerce # revert to revision 1

# Validate chart without installing (lint + dry-run):
helm lint ./product-service-chart
helm template product-service ./product-service-chart --namespace ecommerce
# helm template renders YAML without applying - review before install
```

HPA + Helm Chart for product-service

20 min in-session + homework · Builds on S14-S15 manifests · Grading: Feature 70% + Tests 20% + Quality 10%



Add HPA

k8s/product-service/hpa.yaml — 70% CPU threshold, min 2 / max 10 (5 min)



Create Helm Chart

Chart.yaml + values.yaml + templates/deployment.yaml + serviceaccount/role (12 min)



Verify Helm Lifecycle

helm lint, helm install, helm list, helm rollback (3 min)

Acceptance Criteria

- ✓ HPA: `kubectl get hpa` shows `product-service-hpa` with `MAXPODS: 10`
- ✓ Helm chart: `Chart.yaml` + `values.yaml` + `templates/` exist and `helm lint` passes
- ✓ `helm install` succeeds — `kubectl get pods -n ecommerce` shows pods
- ✓ ServiceAccount `product-service-sa` exists
- ✓ Role: read-only `configmaps/secrets` only (`get` + `list`, no `write`)
- ✓ `helm rollback product-service 1` works — reverts to revision 1
- ✓ Commit: `session-16: add-hpa-and-helm-chart-product-service`

BONUS: `values-dev.yaml` + `values-prod.yaml` with different resource limits · `order-service` Helm chart

What We Added Today

🔗 Capability Added

Autoscaling + Packaged Deployment

product-service auto-scales on CPU $\geq$ 70%. Helm chart is parametrized, versioned, rollback-ready. ServiceAccount + Role enforce least privilege.

🔗 Platform — Phase 2 All Done

S9 🔗 Containerized S10 🔗 Tests S11 🔗 Contract

S12 🔗 Saga Orch. S13 🔗 CI Pipeline

S14 🔗 GitOps S15 🔗 K8s Core S16 🔗 K8s Adv.

🔗 Phase 2 Complete — 8 Sessions · 20 Hours · Full Delivery Pipeline Active

S9 → S16: What Did We Build?

S9: Docker

8 services containerized — reproducible environment

S10: Unit + Integration

Testing Pyramid + TestContainers real PostgreSQL

S11: Contract + Chaos

Pact contracts + WireMock + validated Circuit Breaker

S12: Saga Orchestration

OrderSagaOrchestrator + explicit SagaState control

S13: CI Pipeline

GitHub Actions: test → build → push, every push tested

S14: GitOps + Canary

ArgoCD = Git as Source of Truth, drift detection

S15: K8s Core

Probes self-heal deadlocks, Secrets protect passwords

S16: K8s Advanced

HPA scales automatically, Helm packages + versions

Phase 2 → Phase 3: From Deployment Quality to Enterprise Depth

```
git push → GitHub Actions (S13): test → build image → push to GHCR  
    → Update Git manifest → ArgoCD (S14): detect change → deploy to K8s  
    → K8s (S15-S16): probes gate traffic → HPA scales pods → Helm tracks releases
```

Full automation: code → production in minutes, with audit trail,
rollback, and drift detection.

Phase 3 Preview — Sessions 17-24: Advanced & Enterprise

Distributed Tracing (OpenTelemetry/Zipkin) · CQRS · Event Sourcing · OAuth2/Keycloak Security · Istio Service Mesh · Performance & Load Testing

5 Errors You Will See This Session

HPA shows TARGETS: <unknown>/70%

Metrics Server not installed — minikube addons enable metrics-server

helm install: 'cannot re-use a name'

A release already exists — helm upgrade instead, or helm uninstall first

Helm template renders empty ServiceAccount

Check values.yaml structure: serviceAccount.create: true (nested correctly)

HPA scales to maxReplicas immediately

Add behavior.scaleUp.stabilizationWindowSeconds: 60 to avoid over-reacting

Role created but pod gets 'Forbidden'

Check RoleBinding subjects match ServiceAccount name + namespace exactly



 **PHASE 2 COMPLETE**

8 Sessions · 20 Hours · Full Delivery Pipeline Active

S9 Docker 

S10 Tests 

S11 Contract 

S12 Saga 

S13 CI 

S14 GitOps 

S15 K8s 

S16 K8s Adv 

Phase 3 begins Wednesday — Observability, CQRS, API Security, Service Mesh

Session 17: Distributed Tracing with OpenTelemetry + Zipkin